Coding Fourier Series

Kuba Kane

Aim of the code

1. Plot our periodic function

2. Plot the Fourier series representing that function

Plotting functions

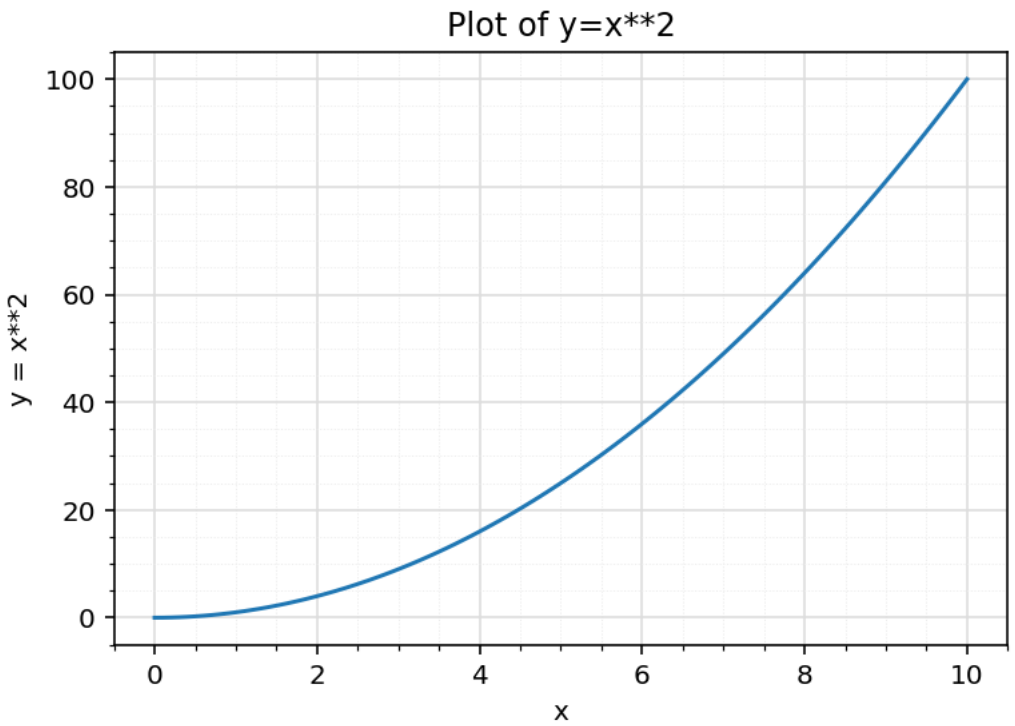
```python
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0,10,100) #from 0 to 10 in steps of 100
y = x**2


fig = plt.figure()
axes = fig.add_subplot(111)
axes.plot(x, y)

axes.set_title('Plot of y=x**2')
axes.set_xlabel('x')
axes.set_ylabel('y = x**2')

axes.grid(which='major', color='#DDDDDD', linewidth=0.8)
axes.grid(which='minor', color='#EEEEEE', linestyle=':', linewidth=0.5)
axes.minorticks_on()
plt.savefig("even_process_Cm.png", dpi=600)
plt.show()
plt.close()
```



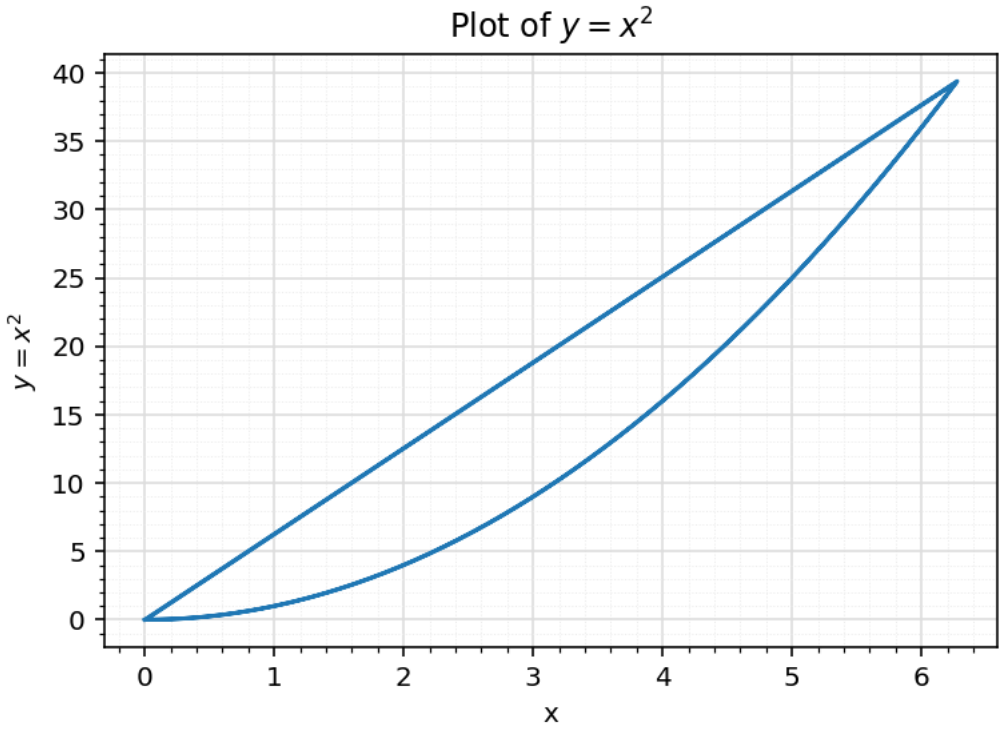
Kuba Kane

Plotting periodic functions

- Fourier series are periodic so we must ensure they are repeating after some fixed Interval

- E.g. make the function repeat every 2π

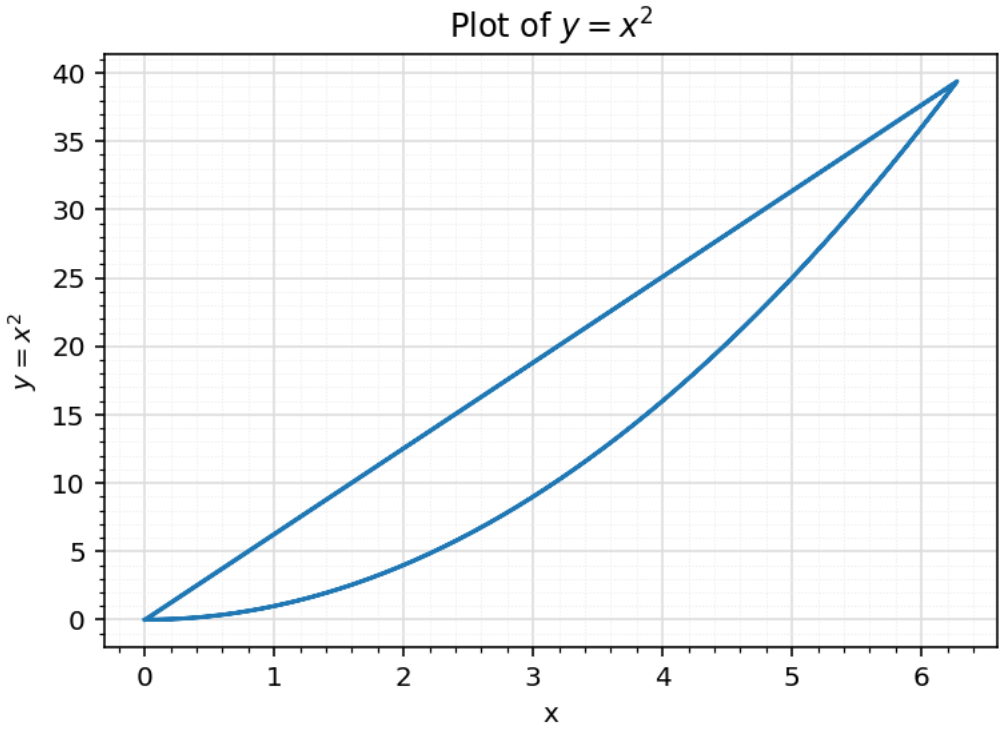
Plotting periodic functions



Why is this wrong? What has happened?

Kuba Kane

Plotting periodic functions



Why is this wrong? What has happened?

```
x = np.linspace(0,4*np.pi,1000)
x = x % (2*np.pi)
y = x**2
```

Kuba Kane

Plotting periodic functions

0. 0.12693304 0.25386607 0.38079911 0.50773215 0.63466518
0.76159822 0.88853126 1.01546429 1.14239733 1.26933037 1.3962634
1.52319644 1.65012947 1.77706251 1.90399555 2.03092858 2.15786162
2.28479466 2.41172769 2.53866073 2.66559377 2.7925268 2.91945984
3.04639288 3.17332591 3.30025895 3.42719199 3.55412502 3.68105806
3.8079911 3.93492413 4.06185717 4.1887902 4.31572324 4.44265628
4.56958931 4.69652235 4.82345539 4.95038842 5.07732146 5.2042545
5.33118753 5.45812057 5.58505361 5.71198664 5.83891968 5.96585272
6.09278575 6.21971879 0.06346652 0.19039955 0.31733259 0.44426563
0.57119866 0.6981317 0.82506474 0.95199777 1.07893081 1.20586385
1.33279688 1.45972992 1.58666296 1.71359599 1.84052903 1.96746207
2.0943951 2.22132814 2.34826118 2.47519421 2.60212725 2.72906028
2.85599332 2.98292636 3.10985939 3.23679243 3.36372547 3.4906585
3.61759154 3.74452458 3.87145761 3.99839065 4.12532369 4.25225672
4.37918976 4.5061228 4.63305583 4.75998887 4.88692191 5.01385494
5.14078798 5.26772102 5.39465405 5.52158709 5.64852012 5.77545316
5.9023862 6.02931923 6.15625227 0.

There is a sudden jump

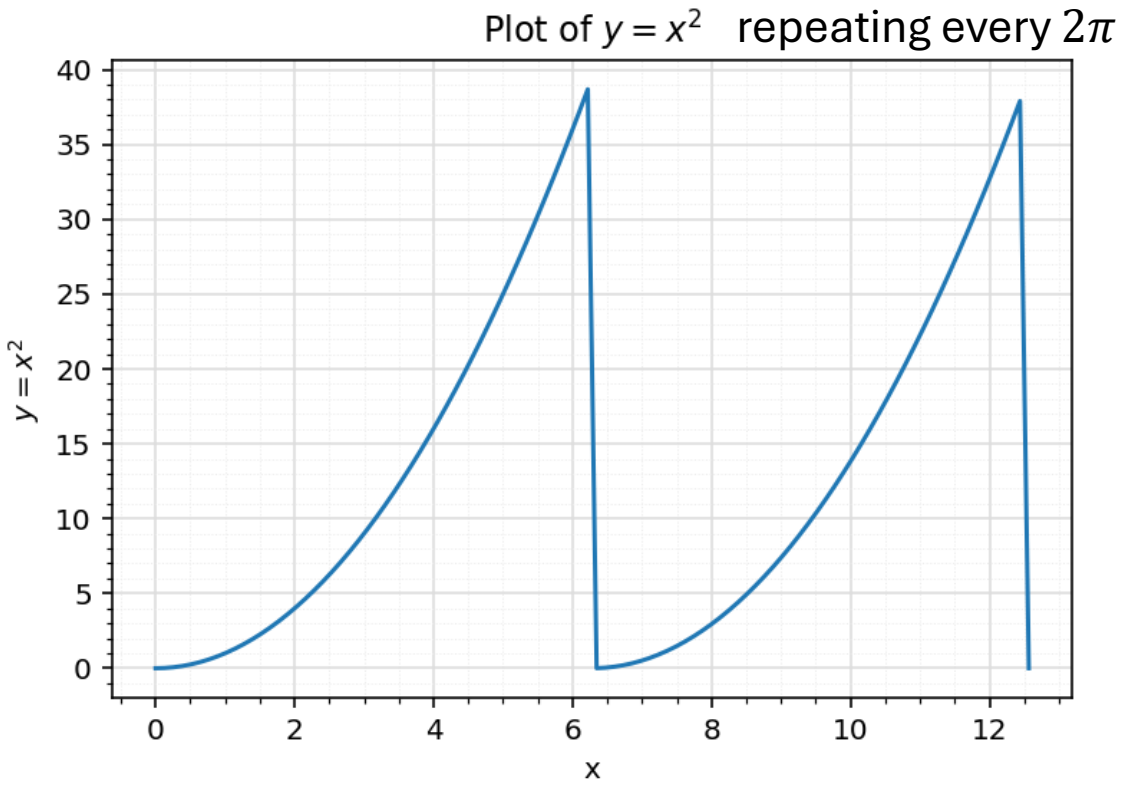
Kuba Kane

Plotting periodic functions

Solution: order data in ascending order manually, or:

```
x = np.linspace(0,4*np.pi,100) #from 0 to 10 in steps of 100
y = (x % (2*np.pi))**2
```

Plotting periodic functions



Kuba Kane

Fourier series to code

$$f(t) = \frac{1}{2}a_0 + \sum_{n=1}^{\infty} a_n \cos(n\omega t) + b_n \sin(n\omega t)$$

$$a_0 = \frac{\omega}{\pi}\int_{-\frac{\pi}{\omega}}^{\frac{\pi}{\omega}} f(t)dt$$

$$a_n = \frac{\omega}{\pi}\int_{-\frac{\pi}{\omega}}^{\frac{\pi}{\omega}} f(t)\cos(n\omega t)\,dt$$

$$b_n = \frac{\omega}{\pi}\int_{-\frac{\pi}{\omega}}^{\frac{\pi}{\omega}} f(t)\sin(n\omega t)\,dt$$

Kuba Kane

Coding integrals

- Quad function
https://docs.scipy.org/doc/scipy/reference/generated/scipy.integrate.quad.html

- Requires scipy

Quad function

quad(*func, a, b*, *args*=*()*, *full_output*=*0*, *epsabs*=*1.49e-08*, *epsrel*=*1.49e-08*, *limit*=*50*, *points*=*None*, *weight*=*None*, *wvar*=*None*, *wopts*=*None*, *max p1*=*50*, *limlst*=*50*, *complex_func*=*False*)

Quad function

Calculate $\int_0^4 x^2 dx$ and compare with an analytic result

```
>>> from scipy import integrate
>>> import numpy as np
>>> x2 = lambda x: x**2
>>> integrate.quad(x2, 0, 4)
(21.333333333333332, 2.3684757858670003e-13)
>>> print(4**3 / 3.)  # analytical result
21.3333333333
```

Quad function

Returns:

y : *float*

The integral of func from *a* to *b*.

abserr : *float*

An estimate of the absolute error in the result.

infodict : *dict*

A dictionary containing additional information.

message

A convergence message.

explain

Appended only with 'cos' or 'sin' weighting and infinite integration limits, it contains an explanation of the codes in infodict['ierlst']

Kuba Kane

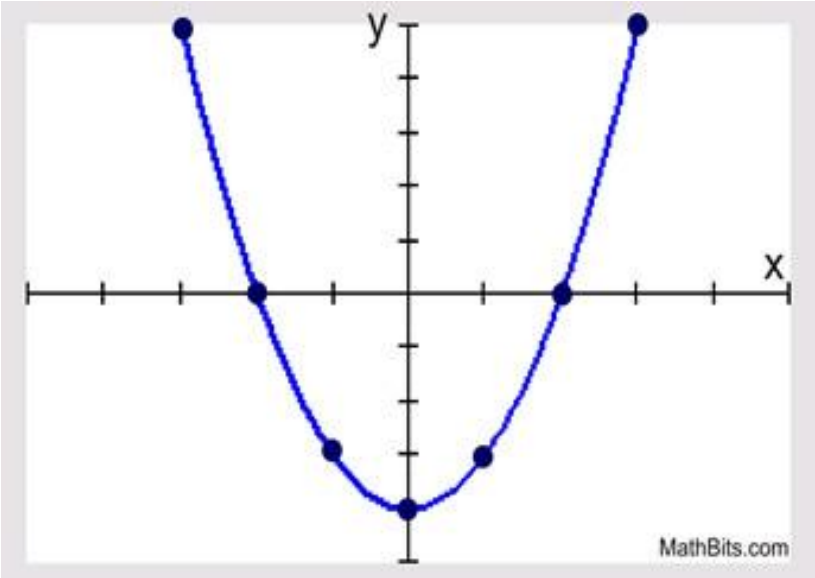
Coding a_0

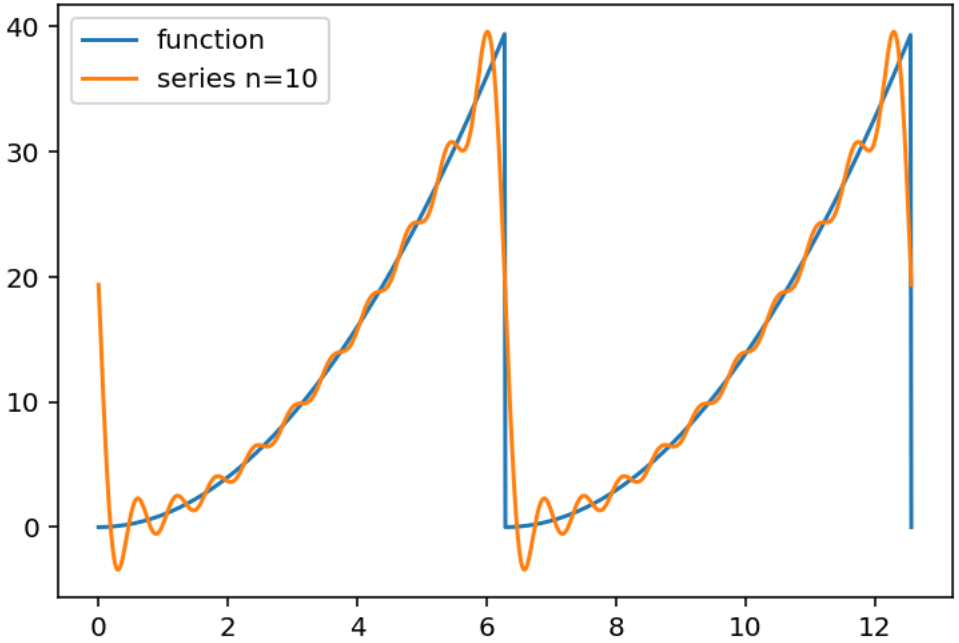
- Start with coefficients…

$$\boxed{a_0 = \frac{\omega}{\pi}\int_{-\frac{\pi}{\omega}}^{\frac{\pi}{\omega}} f(t)dt}$$

Coding a_0

What about the limits? Our function needs to be periodic → we can choose where to bound the function & repeat it





Kuba Kane

Coding Fourier series

So we choose $0 \rightarrow 2\pi$

But then we need to change our function so that it repeats every 2π...

```
x = x % (2*np.pi)
```

$$T = 2\pi, \omega = \frac{2\pi}{T} = 1$$

Coding a_0

$$a_0 = \frac{\omega}{\pi}\int_{-\frac{\pi}{\omega}}^{\frac{\pi}{\omega}} f(t)dt$$

- $\frac{\omega}{\pi} = \frac{1}{\pi}$
- $Choose\ 0 \rightarrow 2\pi$

```
a_0 = integrate.quad(func,0,2*np.pi)[0]/(  np.pi)
```

Coding a_0

- Test it:

```
#Plotting a_0
import numpy as np
import scipy.integrate as integrate

def function(x):
    x = x**2
    return x


a_0 = integrate.quad(function,0,2*np.pi)[0]/(2*np.pi)

print(a_0)
```

a_0_coeff.py

Coding a_n, b_n

- Now we compute rest of coefficients

$$\boxed{a_n = \frac{\omega}{\pi}\int_{-\frac{\pi}{\omega}}^{\frac{\pi}{\omega}} f(t)\cos(n\omega t)\,dt}$$

$$\boxed{b_n = \frac{\omega}{\pi}\int_{-\frac{\pi}{\omega}}^{\frac{\pi}{\omega}} f(t)\sin(n\omega t)\,dt}$$

- We can ignore $n = 0$ terms since this is already accounted for by the constant a_0 (as specified by our sum)

$$f(t) = \frac{1}{2}a_0 + \sum_{n=1}^{\infty} a_n\cos(n\omega t) + b_n\sin(n\omega t)$$

Coding a_n, b_n

- Recall we set $\omega = 1$ → equations simplify:

$$f(t) = \frac{1}{2}a_0 + \sum_{n=1}^{\infty} a_n \cos(nt) + b_n \sin(nt)$$

$$a_n = \frac{1}{\pi}\int_0^{2\pi} f(t)\cos(nt)\,dt$$

$$b_n = \frac{1}{\pi}\int_0^{2\pi} f(t)\sin(nt)\,dt$$

$$a_0 = \frac{1}{\pi}\int_0^{2\pi} f(t)dt$$

Coding a_n

- Problem: x is not defined

```
#A_n coefficient
import numpy as np
import scipy.integrate as integrate

def function(x):
    x = x**2
    return x

A_n = integrate.quad(function(x) * np.sin(n*x), 0, 2*np.pi)[0] / np.pi

print(A_n)
```

A_n_coeff.py

Coding a_n

- Solution: use lambda function

```
#A_n coefficient
import numpy as np
import scipy.integrate as integrate

def function(x):
    x = x**2
    return x

A_n = integrate.quad(lambda x: function(x) * np.sin(n*x), 0, 2*np.pi)[0] / np.pi

print(A_n)
```

A_n_coeff.py

Lambda function

- Allows us to pass arguments which haven't been defined yet

Summarize argument `a`, `b`, and `c` and return the result:

```
x = lambda a, b, c : a + b + c
print(x(5, 6, 2))
```

- We can use this to simplify how we will code the integrals

Coding a_n

- Problem: n is not defined → needs to be user input

```
#A_n coefficient
import numpy as np
import scipy.integrate as integrate

def function(x):
    x = x**2
    return x

A_n = integrate.quad(lambda x: function(x) * np.sin(n*x), 0, 2*np.pi)[0] / np.pi

print(A_n)
```

A_n_coeff.py

Coding a_n

```python
#A_n coefficient
import numpy as np
import scipy.integrate as integrate

def function(x):
    x = x**2
    return x

n = int(input("How many terms in Fourier series? "))
A_n = integrate.quad(lambda x: function(x) * np.sin(n*x), 0, 2*np.pi)[0] / np.pi

print(A_n)
```

A_n_coeff.py

Coding b_n

```python
#A_n coefficient
import numpy as np
import scipy.integrate as integrate

def function(x):
    x = x**2
    return x

n = int(input("How many terms in Fourier series? "))
A_n = integrate.quad(lambda x: function(x) * np.cos(n*x), 0, 2*np.pi)[0] / np.pi
B_n = integrate.quad(lambda x: function(x) * np.sin(n*x), 0, 2*np.pi)[0] / np.pi

print(A_n, B_n)
```

A_n_coeff.py

Coding a_n, b_n

- But have we treated n correctly?

$$f(t) = \frac{1}{2}a_0 + \sum_{n=1}^{\infty} a_n \cos(nt) + b_n \sin(nt)$$

$$a_n = \frac{1}{\pi}\int_0^{2\pi} f(t)\cos(nt)\,dt$$

$$b_n = \frac{1}{\pi}\int_0^{2\pi} f(t)\sin(nt)\,dt$$

$$a_0 = \frac{1}{\pi}\int_0^{2\pi} f(t)dt$$

Coding a_n, b_n

- If user enters n=10, it tells us the term a_{10} or b_{10}
- But we need to sum over all terms from n = 1 to n = 10

- Requires defining another function

Coding a_n, b_n

- If user enters n=10, it tells us the term a_{10} or b_{10}
- But we need to sum over all terms from n = 1 to n = 10

- Requires defining another function
- Hint: use an array to store all the coefficients

Coding a_n, b_n

```python
#An and Bn coefficients sum

import numpy as np
import scipy.integrate as integrate

def function(x):
    x = x**2
    return x

n = int(input("How many terms in Fourier series? "))

A_coefficients = np.array([])
B_coefficients = np.array([])

for i in range(n):
    A_n = integrate.quad(lambda x: function(x) * np.cos(i*x), 0, 2*np.pi)[0] / np.pi
    B_n = integrate.quad(lambda x: function(x) * np.sin(i*x), 0, 2*np.pi)[0] / np.pi

    A_coefficients = np.append(A_coefficients, A_n)
    B_coefficients = np.append(B_coefficients, B_n)

A_total = np.sum(A_coefficients)
B_total = np.sum(B_coefficients)

print(A_total, B_total)

print("A_coefficients", A_coefficients)
```

a_n_b_n_coeffs.py

Coding a_n, b_n

- 3 parts:

1) Setting up the arrays
2) Appending to the arrays (with the a_n, b_n coefficients)
3) Summing over all arrays

Coding a_n, b_n

- **Important:**

This sums over range(n) = 0, 1, 2, … up to the user inputted value – 1

E.g. if user enters 5: range(n) = 0, 1, 2, 3, 4

Our definition of sum goes from n=1 to n=5 so we can use (i+1) to ensure the correct sum…

$$f(t) = \frac{1}{2}a_0 + \sum_{n=1}^{\infty} a_n \cos(nt) + b_n \sin(nt)$$

Coding a_n, b_n

a_n_b_n_coeffs.py

```python
#An and Bn coefficients sum

import numpy as np
import scipy.integrate as integrate

def function(x):
    x = x**2
    return x

n = int(input("How many terms in Fourier series? "))

A_coefficients = np.array([])
B_coefficients = np.array([])

for i in range(n):
    A_n = integrate.quad(lambda x: function(x) * np.cos((i+1)*x), 0, 2*np.pi)[0] / np.pi
    B_n = integrate.quad(lambda x: function(x) * np.sin((i+1)*x), 0, 2*np.pi)[0] / np.pi

    A_coefficients = np.append(A_coefficients, A_n)
    B_coefficients = np.append(B_coefficients, B_n)

A_total = np.sum(A_coefficients)
B_total = np.sum(B_coefficients)

print(A_total, B_total)

print("A_coefficients", A_coefficients)
```

Coding a_n, b_n

- But we do not want sum of a_n and b_n...

$$f(t) = \frac{1}{2}a_0 + \sum_{n=1}^{\infty} a_n \cos(nt) + b_n \sin(nt)$$

$$a_n = \frac{1}{\pi}\int_0^{2\pi} f(t)\cos(nt)\,dt$$

$$b_n = \frac{1}{\pi}\int_0^{2\pi} f(t)\sin(nt)\,dt$$

$$a_0 = \frac{1}{\pi}\int_0^{2\pi} f(t)dt$$

Coding a_n, b_n

- a_n, b_n need to be summed over for each n, mutliplied by cos and sin (also functions of n) respectively

$$f(t) = \frac{1}{2}a_0 + \sum_{n=1}^{\infty} a_n \cos(nt) + b_n \sin(nt)$$

Coding a_n, b_n

- How to do this in code?
- Create an array for each term, e.g. $a_1 \cos(1t) + b_1 \sin(1t)$ would be first entry in array

$$f(t) = \frac{1}{2}a_0 + \sum_{n=1}^{\infty} a_n \cos(nt) + b_n \sin(nt)$$

Coding a_n, b_n

- What is wrong with this code?

```
#Fourier sum term
import numpy as np
import scipy.integrate as integrate

def function(x):
    x = x**2
    return x

n = int(input("How many terms in Fourier series? "))

A_coefficients = np.array([])
B_coefficients = np.array([])
sum_term = np.array([])

for i in range(n):
    A_n = integrate.quad(lambda x: function(x) * np.cos(i*x), 0, 2*np.pi)[0] / np.pi
    B_n = integrate.quad(lambda x: function(x) * np.sin(i*x), 0, 2*np.pi)[0] / np.pi

    total_term = lambda x: A_n * np.cos(i*x) +  B_n * np.sin(i*x)
    sum_term = np.append(sum_term, total_term)

print(sum_term)
```

Kuba Kane

Coding a_n, b_n

- total_term is a lambda function, not a number or array
- Lambda function returns a function…

```
How many terms in Fourier series? 10
[<function <lambda> at 0x000002387A3B67A0>
 <function <lambda> at 0x000002387A3B6840>
 <function <lambda> at 0x000002387A3B5DA0>
 <function <lambda> at 0x000002387A3B63E0>
 <function <lambda> at 0x000002387A3B6700>
 <function <lambda> at 0x000002387A3B6AC0>
 <function <lambda> at 0x000002387A3B6C00>
 <function <lambda> at 0x000002387A3B6A20>
 <function <lambda> at 0x000002387A3B6D40>
 <function <lambda> at 0x000002387A3B6660>]
```

Coding a_n, b_n

- We don't have this issue when defining a_n since we are using the quad function, which takes a function as an input
- The quad function therefore deals with it for us

Coding a_n, b_n

- We do not want to use quad again – this is for integrals only

- So when finding the sum term $\sum_{n=1}^{\infty} a_n \cos(nx) + b_n \sin(nx)$ we need to specify **what x is**

Coding a_n, b_n

- What are we trying to do?

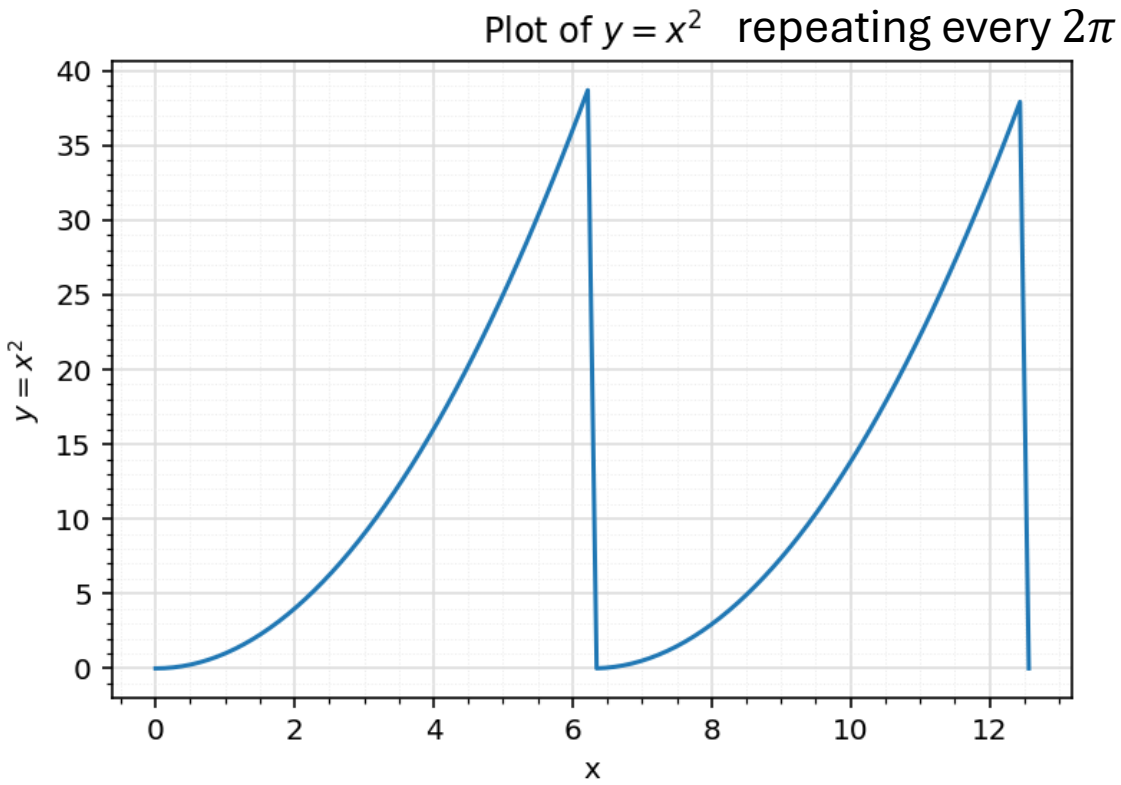
We want to <u>plot</u> the Fourier series representation of a periodic function

- So far we have been using a test function:

```
def function(x):
    x = x**2
    return x
```

Coding a_n, b_n

- We haven't needed to specify x because of the quad function
- We can now specify for what x values to plot our function for
- We can choose any range – let's go with $0 \rightarrow 4\pi$



Coding a_n, b_n

- So we can use the same linspace as before:

```
x = np.linspace(0,10,100) #from 0 to 10 in steps of 100
y = x**2
```

Coding a_n, b_n

- **Function 1:** define the function to plot
- **Function 2:** output a_0, a_n, b_n using quad function
- **Function 3:** take in a_0, a_n, b_n and output the Fourier series for each of the x values in our linspace

- Try coding these functions

Coding a_n, b_n

- Things to watch out for:

1. The function (e.g. x^2) should be periodic so remember we need to use the remainder function so that the output is periodic
2. Use np.full to generate an array full of a_0 values, to which you can add the “sum term” in the Fourier series
3. You will need to use a function within a function – call on the function generating a_0, a_n, b_n within the function generating the plot

Function 1

- This is the same as our plot before, but it needs to all be underneath a function
- It should take x as an input and give us y
- It is the function we want to plot

Function 2

- We want to output a_n, b_n, a_0 for an n which the user chooses (number of terms in the series)

- a_n, b_n can be outputted as an array storing the coefficients for each n

- a_0 is just one value

Function 3

- This is the function which will actually make our plot

- We want to plot two lines:
  1. The y function as a smooth curve
  2. The Fourier series of that curve

Function 3

- For (1), we need the actual y function we have defined to be an input, so that we can plot it

- For (2), we must take n as an input so that the user can choose this
- We can get the coefficients of the Fourier series by calling function 2 within function 3

Function 3

- To evaluate

$$f(t) = \frac{1}{2}a_0 + \sum_{n=1}^{\infty} a_n \cos(nt) + b_n \sin(nt)$$

we need to sum over a_n, b_n multiplied by cos and sin (also dependent on n) respectively

- We can initiate an empty array with $\frac{1}{2}a_0$ and then add the 2nd term (the summation term) to get $f(x)$

Function 3

- Use np.full to generate an array filled with a_0 whose length needs to be equal to the length of x (in order for Python to plot it)
- Recall we have a_n, b_n as arrays so we can extract each element one by one, and multiply it by cos & sin terms as per the definition

- For each n, we add to the series the nth term (from extracting the nth element from a_n, b_n)
- This means each entry in the series (corresponding to each x) has the summation term added to it
- We therefore have f(x) for each x in a new element of the array

Function 3

- This is exactly what we want! We now have a corresponding f(x) value for each x value
- We can now plot this in a similar manner to how we did before
- See final code **fourier_series_plot.py**

Coding a_n, b_n

```
def fourier_series_plot(function, n):
    x = np.linspace(0,4*np.pi,1000) #0 to 4pi in steps of 1000
    y = function(x)

    a_0, a_coefs, b_coefs = series_terms(function, n)
    print("a_0 =: ", a_0)
    print("a_coefs (sin coeffs) = : ", a_coefs)
    print("b_coefs (cos coeffs) = : ", b_coefs)

    plt.plot(x, y, label = "function") #plot of original func

    series = np.full(1000, a_0) #create array where all terms take the value
    #of a_0 (1000 since there are 1000 x steps)

    for i in range(n):
        series += a_coefs[i]*np.cos((i+1)*x) + b_coefs[i]*np.sin((i+1)*x)
        #print(series)

    plt.plot(x, series, label = "series n={}".format(n))
    plt.legend()


n = int(input("How many terms in Fourier series? "))
fourier_series_plot(func, n)
```

Kuba Kane